

Database Systems

- A **Database System** is a collection of **interrelated data** and a set of programs (called **DBMS**) to access and manage that data efficiently.
- It provides an environment that is **both convenient and efficient** for storing, retrieving, and managing data.

Database:

- A database is an organized collection of data, generally stored and accessed electronically from a computer system.
- **Example:**
 - In a **University Database**: tables for students, faculty, courses, exams, and results.
 - In a **Banking Database**: tables for accounts, customers, transactions, loans.

Database Management System (DBMS):

- A DBMS is software that allows users to create, define, manipulate, and maintain the database.
- It acts as a bridge between the database and users/applications.

Example: Oracle, MySQL, PostgreSQL, SQL Server, MongoDB.

Functions of a DBMS

1. **Data Storage Management** – Efficient storage and retrieval of data.
2. **Data Retrieval (Queries)** – Allows complex queries using SQL.
3. **Concurrency Control** – Multiple users can access without conflict.
4. **Transaction Management** – Ensures **ACID** properties (**A**tomicity, **C**onsistency, **I**solation, **D**urability).
5. **Backup & Recovery** – Protects against data loss.
6. **Security Management** – Provides controlled access using authentication.
7. **Integrity Management** – Enforces rules (constraints) like primary key, foreign key.

Advantages of DBMS over File System:

File System	Database System
Data redundancy is high	Redundancy is minimized
Difficult to enforce security	Strong security & authorization
Poor data consistency	High data consistency
Application-dependent data	Data independence provided
Complex backup/recovery	Automatic backup & recovery

View of Data Models

1. Database System Architecture – Levels of Abstraction

Three-level architecture (ANSI/SPARC model):

1. Physical Level (Internal Schema)

- Describes *how* data is actually stored.
- Focus: storage structures, indexes, files.
- Example: Roll_No stored as 4-byte integer.

2. Logical Level (Conceptual Schema)

- Describes *what* data is stored and relationships.
- Focus: entities, attributes, relationships.
- Example: Student table with Roll_No, Name, Dept.

3. View Level (External Schema)

- Describes *part of the database* as seen by a user.
- Different users see different views.
- Example:
 - A student can see marks.
 - A teacher can see marks + attendance.
 - An admin can see fee details.

Data Independence

- **Physical Data Independence:** Change in physical storage does not affect logical schema.
- **Logical Data Independence:** Change in logical schema does not affect user views.

Types of Data Models

1. High-Level (Conceptual) Data Models

- **Definition:** Provide concepts close to how users perceive data in the real world.
- **Purpose:** Easy for non-technical users and database designers to understand.
- **Most Popular: Entity-Relationship (ER) Model.**

Entity-Relationship (ER) Model

- **Entity:** An object in real-world (e.g., Student, Teacher, Course).
- **Attributes:** Properties of entities (e.g., Roll_No, Name, Age).
- **Relationship:** Association between entities (e.g., Student *enrolls in* Course).

Example ER Diagram (explained in text):

- Entity: **Student** (Roll_No, Name, Dept)
- Entity: **Course** (Course_ID, Title, Credits)
- Relationship: **Enrolls** (Student ↔ Course, with attribute Grade)

Used in: Database design stage (conceptual schema).

2. Representational (Logical/Implementation) Data Models

- **Definition:** Provide concepts that may be understood by users but are directly implementable in DBMS.
- **Purpose:** Bridge between high-level design and physical storage.

Types:

1. Relational Model (Most widely used)

- **Proposed by:** E.F. Codd (1970).
- **Data Representation:** Tables (Relations).
 - Rows = Tuples
 - Columns = Attributes
- **Constraints:** Primary key, Foreign key, Domain constraints.
- **Language:** SQL.

2. Hierarchical Model

Data Structure: Tree-like (parent-child relationship).

Each child has only one parent, but parent may have multiple children.

Access: Navigational (need to traverse tree to access data).

Example:

Database for University:

College (root) → Departments → Students.

Limitation: Cannot handle many-to-many relationships easily.

3. Network Model

Data Structure: Graph-like with nodes and edges.

Records are nodes, relationships are edges.

Allows many-to-many relationships.

Uses pointers/links to navigate.

Example:

Student ↔ Course (A student can enroll in many courses, and a course can have many students).

Limitation: Complex to design and implement.

4. Object-Oriented Data Model

Combines database concepts with object-oriented programming (OOP).

Data is represented as objects (with attributes + methods).

Supports inheritance, polymorphism, encapsulation.

Example:

```
Class Student { Roll_No, Name, displayDetails() }
```

```
Inherits from Class Person { Name, Age }
```

Used in applications requiring multimedia, graphics, engineering databases.

Low-Level (Physical) Data Models

Definition: Describe how data is stored physically on hardware.

Details: Record formats, storage structures, indexing, hashing, pointers.

Example: A record might be stored in blocks of 4 KB with B+ tree indexing.

Used by: Database administrators (DBAs) and system programmers.

Importance of Data Models

- Provide **abstraction** → hide storage details from users.
- Help in **designing databases systematically**.
- Allow **data independence**.
- Provide **standards** for communication between users, designers, and developers.
- Ensure **data integrity and consistency**.

Database Languages

Why Languages in DBMS?

- A DBMS provides different languages to:
 - Define database schema.
 - Manipulate data.
 - Control access.
 - Manage transactions.

Types of Database Languages

1. Data Definition Language (DDL)

- Defines database schema (structure).
- Commands:
 - CREATE – create tables.
 - ALTER – modify tables.
 - DROP – delete tables.
 - TRUNCATE – remove all records but keep structure.

Example:

```
CREATE TABLE Employee (  
    Emp_ID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Salary DECIMAL(10,2)  
);
```

2. Data Manipulation Language (DML)

Used to insert, update, delete, and retrieve data.

Types:

Procedural DML: Tells what data + how to get it.

Non-Procedural DML (Declarative): Tells what data only (SQL SELECT).

Example:

```
INSERT INTO Employee VALUES (101, 'Ravi', 50000);
```

```
UPDATE Employee SET Salary = 60000 WHERE Emp_ID = 101;
```

```
DELETE FROM Employee WHERE Emp_ID = 101;
```

```
SELECT * FROM Employee;
```

3. Data Control Language (DCL)

- Controls access to database.
- Commands:
 - GRANT – assign permissions.
 - REVOKE – remove permissions.

Example:

```
GRANT SELECT, UPDATE ON Employee TO user1;
```

```
REVOKE UPDATE ON Employee FROM user1;
```

Transaction Control Language (TCL)

- Manages database transactions.
- Commands:
 - COMMIT – save permanently.
 - ROLLBACK – undo.
 - SAVEPOINT – mark a point in transaction.

 Example:

```
BEGIN TRANSACTION;
```

```
UPDATE Employee SET Salary = 70000 WHERE Emp_ID = 101;
```

```
ROLLBACK; -- undo last update
```

DBMS Architecture

- A **Database Management System (DBMS)** is a collection of programs that enable users to create and maintain a database.
- The **DBMS architecture** defines the structure of the database system, how data is stored, how users interact with it, and how requests are processed.
- It provides an **abstraction** of the database, hiding storage and implementation details from users.

The most widely accepted architecture is the **ANSI/SPARC Three-Level Architecture**.

ANSI/SPARC Three-Level Architecture

- Separate the way data is **stored, represented, and viewed by users**.
- Provide **data independence**.

Levels of Architecture:

1. Internal Level (Physical Schema)

- Closest to physical storage.
- Deals with **how data is actually stored** on disks, memory, indexes, pointers.
- Concerns:
 - File organization.
 - Record placement.
 - Use of B+ trees, hashing.
- Example: Roll_No stored as 4-byte integer, records placed sequentially on disk.

2. Conceptual Level (Logical Schema):

- Describes the **logical structure of the entire database**.
- Hides physical storage details.
- Specifies:
 - Entities, attributes, relationships.
 - Constraints (keys, data types).
- Example: A “Student” entity with Roll_No, Name, Dept.

3. External Level (View Schema)

- Closest to end users.
- Describes **how individual users see the data** (subsets of the database).
- Each user can have a different external view.
- Example:
 - A student views marks and attendance.
 - A faculty views salary + subject assignments.
 - An admin views fee records.

Objectives of Three-Level Architecture

- To **separate user applications** from physical database.
- To provide **multiple user views**.
- To achieve **data independence** (physical and logical).
- To make **database design flexible**.

Two-Tier and Three-Tier Architectures:

Apart from ANSI/SPARC, DBMS architecture is also viewed as **client-server based**.

Two-Tier Architecture

- Application directly communicates with DBMS.
- Structure:
 - Client (front-end) → Server (DBMS).
- Example: Banking applications.

Limitation: Not scalable, poor security for large systems.

Three-Tier Architecture

- Adds a **middle layer** (application server).
- Structure:
 - Client (Presentation Layer) → Application Server (Business Logic) → Database Server (DBMS).
- Example: Web applications, ERP systems.

Advantages: Scalability, better security, load balancing.

Database Users

- A database system has **different categories of users**, each with different roles, rights, and privileges.
- Classification depends on **technical expertise** and **interaction with the DBMS**.

Types of Users

1. Database Administrator (DBA)

- **Role:** Central authority controlling the database.
- **Responsibilities:**
 - Schema definition (DDL).
 - Storage structure & access method definition.
 - Granting/revoking permissions.
 - Performance tuning.
 - Backup & recovery.
 - Security enforcement.
- **Example:** System admin in a university managing student records.

2. Database Designers

- **Role:** Identify data requirements and design database schema.
- **Responsibilities:**
 - Use **ER diagrams** to model entities, attributes, relationships.
 - Ensure data constraints and normalization.
- **Example:** A software engineer designing schema for an e-commerce platform.

3. Application Programmers

- **Role:** Develop programs that interact with DBMS using SQL/embedded languages.
- **Responsibilities:**
 - Write stored procedures, triggers.
 - Develop forms, reports.
- **Example:** Developer writing code for online railway reservation.

4. End Users

- Final users who query or update the database.

Types:

- **Naïve Users:** Use pre-written programs (e.g., ATM user).
- **Casual Users:** Occasionally access DB using queries (e.g., manager checking reports).
- **Sophisticated Users:** Use advanced tools like analytics (e.g., data scientist).
- **Standalone Users:** Use personal databases (e.g., MS Access).

Data Independence

- **Definition:** Data independence is the ability to modify a schema definition at one level without affecting schema at the next higher level.
- Ensures **flexibility** and **isolation** between database design layers.

Types of Data Independence

1. Physical Data Independence

- Ability to change **internal schema** without changing logical schema.
- Example:
 - Change storage structure from heap file to B+ tree.
 - Move data from SSD to cloud storage.

User queries remain unaffected.

2. Logical Data Independence

- Ability to change **conceptual schema** without affecting external views.
- Example:
 - Adding new attribute "Email" in Student entity.
 - Splitting table "Employee" into "Permanent" and "Contract".

Applications using old views still work.

Importance of Data Independence

- Provides **data abstraction**.
- Facilitates **database evolution** without affecting applications.
- Reduces **maintenance cost**.
- Increases **system flexibility**.

Entity–Relationship (ER) Modelling

- The **Entity–Relationship (ER) Model** was introduced by **Peter Chen in 1976**.
- It is a **high-level conceptual data model** used for database design.
- Purpose: Provide a simple, visual way to represent **entities, attributes, and relationships**.
- Forms the **blueprint** of the database before conversion into a relational schema.

Components of ER Model

1. Entity

- An **object** in the real world that can be uniquely identified.
- Types:
 - **Strong Entity**: Has a primary key.
 - Example: Student (Roll_No, Name, Dept).
 - **Weak Entity**: Cannot be uniquely identified without a related entity.
 - Example: Dependent (Name, Age) depends on Employee.

2. Attributes

- Properties/characteristics of entities.
- Types:
 1. **Simple Attribute** → Cannot be divided (Name, Age).
 2. **Composite Attribute** → Can be divided (FullName → FirstName + LastName).
 3. **Derived Attribute** → Derived from other values (Age from DOB).
 4. **Multivalued Attribute** → Can have multiple values (Phone Numbers).
 5. **Key Attribute** → Uniquely identifies an entity (Roll_No).

3. Relationships

- Association among entities.
- Types (based on cardinality):

1. One-to-One (1:1)

- Example: Each student has one ID card.

2. One-to-Many (1:N)

- Example: A department has many students.

3. Many-to-Many (M:N)

- Example: A student can enroll in many courses, and a course has many students.

4. Relationship Attributes

- Sometimes, relationships have attributes.
- Example:
 - **Enrolls** relationship between Student and Course may have an attribute *Grade*.

5. Weak Entities and Identifying Relationships

- A **weak entity** depends on a **strong entity** for identification.
- Example: Dependent depends on Employee.
- Represented with **double rectangle** (weak entity) and **double diamond** (identifying relationship).

Enhanced ER (EER) Model

- Extension of ER model to support additional concepts.

1. Specialization

- Top-down process.
- An entity is divided into sub-entities (subclasses).
- Example: Employee $\rightarrow$ { Teacher, Clerk, Manager }.

2. Generalization

- Bottom-up process.
- Several entities are combined into a higher-level entity.
- Example: Car, Bike → Vehicle.

3. Aggregation

- Represents **relationship of relationships**.
- Example: A "Project" assigned to "Employee" is supervised by a "Manager".

Advantages of ER Model

The Entity–Relationship (ER) model has many strengths because it provides a clear, structured, and visual representation of data.

1. High-Level Conceptual Design

- ER diagrams represent data at a **conceptual level**, which is independent of physical storage.
- Makes it easy for non-technical users (managers, domain experts) to understand the system.

2. Visual Representation

- ER diagrams provide a **graphical view** of entities, attributes, and relationships.
- Symbols like rectangles, ellipses, and diamonds make the design intuitive.
- Easy to present during discussions with stakeholders.

3. Communication Bridge

- Provides a **common platform** for communication between:
 - End users (who know business rules),
 - Database designers, and
 - Developers.
- Helps in reducing misunderstanding.

4. Systematic Approach

- Follows a **step-by-step approach** (Identify entities → Attributes → Relationships → Diagram).
- Makes the design more **organized and structured**.

5. Strong Foundation for Relational Model

- ER diagrams can be **easily converted** into relational schema (tables, keys, relationships).
- Example: Student–Course ER diagram → Student, Course, and Enroll relations.

6. Flexibility in Representation

- Supports concepts like **weak entities, multivalued attributes, specialization, generalization, and aggregation**.
- Can represent simple as well as **complex real-world scenarios**.

7. Supports Data Integrity

- Clearly defines **keys, relationships, and constraints**, which helps in ensuring consistency.

8. Early Problem Detection

- During ER modelling, inconsistencies, redundancies, and missing entities can be detected **before implementation**.
- Saves cost and time.

9. Easier Maintenance and Modification

- Since the ER diagram is visual and modular, changes in business rules can be **easily incorporated**.

Limitations of ER Model

Even though the ER model is powerful, it also has some weaknesses.

1. Not Suitable for Large Systems

- For complex databases with hundreds of entities and relationships, the ER diagram becomes **too large and unreadable**.

2. Lack of Standardization

- Different designers use **different notations** (Chen's notation, Crow's foot notation, UML).
- This creates confusion and inconsistency.

3. Limited to Structural Aspects

- ER model focuses only on **structural relationships** (entities, attributes, relationships).
- It **does not represent operations, business rules, or dynamic behavior** of data.
- Example: Cannot model how salary is calculated or when a student can enroll.

4. Cannot Handle Unstructured Data Well

- Not suitable for modern applications that use multimedia, images, videos, or semi-structured data (XML/JSON).

5. No Standard Query Support

- ER model is just for design.
- It does not provide a **query language** (like SQL in relational model).
- Hence, it cannot be used for implementation directly.

6. Does Not Show Data Manipulation

- Only shows entities and relationships.
- Does not specify how **data will be inserted, updated, or deleted**.

7. Difficulty in Mapping Many-to-Many Relationships

- ER diagrams allow many-to-many (M:N) relationships, but relational databases do not support them directly.
- Requires conversion using an **intermediate (associative) entity**, which complicates design.

8. Abstract – Lacks Detail

- ER models are **too high-level**.
- Do not capture low-level implementation details (e.g., indexes, storage, file structures).

9. Not Always Intuitive for End Users

- For non-technical people, advanced ER concepts like **specialization, aggregation, weak entities** may be confusing.

Relational Model

- Proposed by **E.F. Codd in 1970**.
- Most widely used logical data model.
- Represents data in **tables (relations)**.
- Each table has **rows (tuples)** and **columns (attributes)**.

Basic Concepts of Relational Model

1. Relation (Table)

- A relation is a **two-dimensional table**.
- Properties:
 - Each relation has a unique name.
 - Rows = Tuples.
 - Columns = Attributes.
 - Order of rows and columns does not matter.
 - No duplicate tuples.

2. Attributes and Domains

- **Attribute** = column name.
- **Domain** = set of valid values for an attribute.
- Example: $\text{Dept} \in \{\text{CSE}, \text{ECE}, \text{IT}, \text{ME}\}$.

3. Keys

- Keys are attributes that uniquely identify tuples.

Types:

1. **Super Key**: Any set of attributes that uniquely identify tuples.
2. **Candidate Key**: Minimal super key.
3. **Primary Key**: Selected candidate key.
4. **Foreign Key**: Attribute in one table referring to primary key of another.
5. **Composite Key**: Combination of attributes.

4. Integrity Constraints

- Rules to maintain **data correctness and consistency**.
- Types:
 1. **Domain Constraint**: Value must belong to domain.
 2. **Entity Integrity Constraint**: Primary key cannot be NULL.
 3. **Referential Integrity Constraint**: Foreign key must match primary key in another table.

Relational Algebra

- Procedural query language.
- Basic operations:
 1. **Select (σ)** – filters rows.
 2. **Project (π)** – selects columns.
 3. **Union ($\cup$)** – combines relations.
 4. **Set Difference ($-$)**.
 5. **Cartesian Product ($\times$)**.
 6. **Rename (ρ)**.
 7. **Join ($\bowtie$)**.

Advantages of Relational Model

1. Simplicity

- Data is represented in **tables (relations)**, which are intuitive and easy to understand.
- Rows = Tuples (records), Columns = Attributes (fields).
- Example: A STUDENT table with columns Roll_No, Name, Dept looks like a real-world spreadsheet.
- **Why it's an advantage?** → Easy for both programmers and non-programmers to grasp.

2. Data Independence

- Relational model supports **logical and physical data independence**.
- Physical changes (e.g., using B+ tree indexes, different file storage) do not affect the logical schema.
- Logical changes (e.g., adding new attributes to a relation) usually do not affect user applications.
- **Why it's an advantage?** → Makes the system **flexible and easier to maintain**.

3. Powerful and Declarative Query Language (SQL)

- Relational model led to the development of **SQL (Structured Query Language)**.
- SQL allows users to **query data declaratively**, i.e., specify *what* they want, not *how* to get it.

4. Data Integrity and Accuracy

- Supports **integrity constraints** that ensure data correctness:
 - **Entity Integrity** → Primary key must be unique and not null.
 - **Referential Integrity** → Foreign key must match primary key in another table.
 - **Domain Constraints** → Attribute values must belong to a defined domain.

- Example: A Roll_No cannot be null, and a student must belong to a valid Dept.
- **Why it's an advantage?** → Ensures **high-quality, consistent, and error-free data**.

5. Flexibility in Data Manipulation

- Users can easily perform **insertion, deletion, and updating** of records without restructuring tables.
- Example: Adding a new student to the STUDENT table requires just one INSERT command.
- **Why it's an advantage?** → The database remains **adaptable** to changing requirements.

6. Reduced Data Redundancy and Better Normalization

- Normalization process removes **duplicate data** and maintains relationships through keys.
- Example: Instead of repeating department details in every student record, we create a separate DEPARTMENT table linked with STUDENT using Dept_ID.
- **Why it's an advantage?** → Saves storage, reduces anomalies, and improves efficiency.

7. Security and Authorization

- Relational databases allow **fine-grained access control**:
 - A user can be given access only to certain tables or certain columns.
 - Example:
 - Accountant can see Fee table but not ExamResults.
 - Teacher can see ExamResults but not Salary details.
- **Why it's an advantage?** → Protects sensitive data and enforces privacy.

8. Multiple Views for Different Users

- Supports **external schemas (views)** so that each user sees only relevant data.
- Example:
 - Student sees only his grades.
 - Teacher sees grades of students in his subject.
- **Why it's an advantage?** → Customization improves usability and security.

9. Support for Large Databases and Transactions

- Relational databases efficiently handle **millions of records** with indexing and optimization.
- They also support **transactions** with ACID properties:
 - **Atomicity** → All or nothing execution.
 - **Consistency** → Ensures database moves from one valid state to another.
 - **Isolation** → Multiple transactions do not interfere.
 - **Durability** → Once committed, data is permanent.
- **Why it's an advantage?** → Ensures **reliability, accuracy, and scalability**.

Introduction to SQL

- **SQL (Structured Query Language)** is the **standard language** used for interacting with relational databases.
- It was developed in the **1970s by IBM** (originally called SEQUEL) and later standardized by **ANSI (1986)** and **ISO (1987)**.
- SQL allows users to **define, manipulate, control, and query** data in a relational model.

Characteristics of SQL

1. **Non-Procedural Language** → Users specify *what* they want, not *how* to retrieve it.
2. **Standardized** → Supported by almost all RDBMSs (Oracle, MySQL, PostgreSQL, SQL Server).
3. **Declarative and Powerful** → Allows simple queries as well as complex joins.
4. **Integration** → SQL can be embedded into high-level languages like Java, Python, and C.

Categories of SQL

1. Data Definition Language (DDL)

- Used to define the structure of database objects.
- Commands:
 - CREATE → Create tables, views, indexes.
 - ALTER → Modify structure of tables.
 - DROP → Delete tables or other objects.

Example:

```
CREATE TABLE STUDENT (  
  Roll_No INT PRIMARY KEY,  
  Name VARCHAR(50),  
  Dept VARCHAR(20)  
);
```

2. Data Manipulation Language (DML)

- Used to manipulate (insert, update, delete) data.
- Commands:
 - INSERT → Add records.
 - UPDATE → Modify records.
 - DELETE → Remove records.

Example:

```
INSERT INTO STUDENT VALUES (101, 'Ravi', 'CSE');
```

3. Data Query Language (DQL)

- Used to retrieve data.
- Command: SELECT

Example:

```
SELECT Name FROM STUDENT WHERE Dept='CSE';
```

4. Data Control Language (DCL)

- Manages access rights and permissions.
- Commands:
 - GRANT → Give permissions.
 - REVOKE → Withdraw permissions.

5. Transaction Control Language (TCL)

- Manages transactions in the database.
- Commands:
 - COMMIT → Save changes.
 - ROLLBACK → Undo changes.
 - SAVEPOINT → Create checkpoints.

Advantages of SQL

- Simple, user-friendly language.
- Portable and supported by all relational databases.
- Can handle large volumes of data.
- Provides security, data integrity, and transaction management.
- Suitable for both **casual users** and **programmers**.

Relational Database Design

- Relational database design is the process of **structuring data into relations (tables)** to ensure:
 - Accuracy
 - Consistency
 - Reduced redundancy
 - Easy retrieval and updates
- Poor database design can lead to **anomalies, redundancy, and inconsistency**.

Steps in Relational Database Design

Step 1: Requirement Analysis

- Collect requirements from users.
- Identify **entities, attributes, and relationships**.
- Example: In a University DB → Entities: Student, Course, Faculty.

Step 2: Conceptual Design (ER Modelling)

- Create an **ER diagram** to represent entities, attributes, and relationships.

Step 3: Conversion to Relational Schema

- Convert ER diagram into tables.
- Example:
 - STUDENT(Roll_No, Name, Dept)
 - COURSE(CourseID, Title, Credits)
 - ENROLL(Roll_No, CourseID, Grade)

Step 4: Normalization

- Process of eliminating redundancy and anomalies.
- Normal Forms:
 - **1NF**: Eliminate repeating groups.
 - **2NF**: Remove partial dependency.
 - **3NF**: Remove transitive dependency.
 - **BCNF**: Stronger form of 3NF.

Step 5: Define Keys and Constraints

- Identify **Primary Keys, Foreign Keys, Candidate Keys**.
- Apply **integrity constraints** (entity, referential, domain).

Step 6: Implementation in SQL

- Use DDL commands to create tables.

Step 7: Testing and Optimization

- Insert sample data.
- Run queries to check correctness.
- Apply indexes, optimize queries for performance.

Importance of Relational Database Design

- Reduces **data redundancy**.
- Ensures **data integrity and consistency**.
- Improves **query performance**.
- Makes database **scalable and maintainable**.
- Provides **better security and access control**.

Database Design Anomalies

(A) Insertion Anomaly

- Cannot insert data because other data is missing.
- Example: Cannot add a new course unless a student enrolls in it.

(B) Update Anomaly

- Updating one record requires updating many others.
- Example: Changing a department name in multiple rows.

(C) Deletion Anomaly

- Deleting a record may remove valuable information.
- Example: Deleting the last student in a department removes department details.

Example: University Database Design

1. **Entities:** Student, Course, Faculty, Department.
2. **ER Model:** Student enrolls in Course, Faculty teaches Course, Course belongs to Department.
3. **Relational Schema:**
 - STUDENT(Roll_No, Name, Dept_ID)
 - COURSE(CourseID, Title, Dept_ID)
 - FACULTY(FacultyID, Name, Dept_ID)
 - DEPARTMENT(Dept_ID, DeptName)
 - ENROLL(Roll_No, CourseID, Grade)

- 4. Normalization:** Ensures no redundant storage of department details in multiple places.

Characteristics of a Good Relational Design

- **Minimal redundancy.**
- **High consistency and integrity.**
- **Scalability** for future expansion.
- **Support for constraints and business rules.**
- **Efficient queries and updates.**
- **Ease of maintenance.**

Challenges in Relational Database Design

- Handling very large datasets.
- Trade-off between normalization and performance.
- Balancing **storage efficiency vs. query speed.**
- Designing for **scalability** in distributed systems.
- Ensuring **security & access control.**

Database Security

- **Database Security** refers to the set of processes, controls, and tools used to protect a database against threats such as unauthorized access, misuse, theft, corruption, or destruction.
- Databases store **critical organizational data** (banking details, academic records, healthcare info, etc.), making them a **prime target for security attacks**.

Goals of Database Security:

1. **Confidentiality** → Only authorized users should access data.
2. **Integrity** → Data should remain accurate and unaltered by unauthorized users.
3. **Availability** → Authorized users must be able to access data whenever required.

Threats to Database Security

1. **Unauthorized Access** → Hackers gaining entry without permission.
2. **SQL Injection Attacks** → Malicious queries altering database operations.
3. **Data Leakage** → Sensitive data exposed due to weak security.
4. **Malware & Viruses** → Infecting database systems.
5. **Denial of Service (DoS)** → Attacks that make the database unavailable.
6. **Insider Threats** → Employees misusing their access rights.

Database Security Mechanisms

(A) Access Control

- **Discretionary Access Control (DAC):**
 - Based on privileges granted by owners.
 - Example: A user can GRANT permissions to others in SQL.
- **Mandatory Access Control (MAC):**
 - Based on security labels (Top Secret, Secret, Confidential).
 - Common in military/government databases.

- **Role-Based Access Control (RBAC):**

- Users assigned roles (Admin, Faculty, Student) with predefined privileges.

(B) Encryption

- Protects data by converting it into unreadable form.
- **Types:**
 - **Data-at-rest encryption** → Protects stored data.
 - **Data-in-transit encryption** → Protects data during communication.
- Example: AES, RSA encryption algorithms.

(C) Authentication & Authorization

- **Authentication:** Verifying identity (passwords, biometrics, tokens).
- **Authorization:** Deciding what actions authenticated users can perform.

(D) Integrity Controls

- Enforce **constraints** (Primary Key, Foreign Key, Check) to prevent invalid data.

(E) Database Auditing

- Tracks and logs user activities to detect unauthorized access.

(F) Backup & Recovery

- Protects against accidental data loss or hardware failure.
- Includes **full backup, incremental backup, and point-in-time recovery**.

Threats to Database Security

(A) Human-based Threats

- **Insider threats** → Employees misusing privileges.
- **Carelessness** → Weak passwords, accidental deletions.
- **Social engineering attacks** → Tricking users into revealing credentials.

(B) Technical Threats

- **SQL Injection** → Attacker inserts malicious SQL commands.
- **Malware / Viruses** → Alter or steal data.
- **Buffer Overflow Attacks** → Exploit memory vulnerabilities in DBMS.
- **Denial of Service (DoS) Attacks** → Flooding system to deny access.

(C) Physical Threats

- Natural disasters (fire, flood, earthquake).
- Hardware theft or failure.

Database Security Policies

1. Least Privilege Principle

- Users get minimum rights required for their role.

2. Separation of Duties

- Divide responsibilities so no single user can misuse power.

3. Need-to-Know Principle

- Users access only the data necessary for their tasks.

4. Defense in Depth

- Multiple layers of security (firewall + access control + encryption).

Challenges in Database Security

- Balancing **security vs. performance** (e.g., encryption slows queries).
- Managing **large number of users** in big organizations.
- **Insider threats** (most dangerous as insiders already have access).
- Ensuring **compliance with legal regulations** (GDPR, HIPAA, PCI DSS).
- Handling **emerging threats** (AI-based attacks, ransomware).

Advantages of Strong Database Security

- Protects sensitive organizational and customer data.
- Prevents financial and reputational losses.
- Ensures compliance with laws and standards.
- Builds trust with clients and stakeholders.
- Reduces risk of insider and outsider attacks.

Limitations

- High cost of implementation and monitoring.
- Performance overhead due to encryption and auditing.
- Complex management for large systems.
- Cannot fully eliminate insider threats.

Transaction Management

- A **transaction** is a sequence of database operations (read/write) treated as a single logical unit of work.
- Example: In a bank transfer:
 - Deduct ₹1000 from A's account.
 - Add ₹1000 to B's account.
 - Both operations must succeed **together** or fail **together**.

Properties of Transactions (ACID)

1. **Atomicity** → Transaction is “all-or-nothing”.
 - If one part fails, the whole transaction rolls back.
2. **Consistency** → Transaction brings database from one valid state to another valid state.
3. **Isolation** → Concurrent transactions should not interfere with each other.
4. **Durability** → Once committed, changes are permanent even if the system crashes.

(A) Atomicity – “All or Nothing”

- Ensures that a transaction is executed **completely or not at all**.
- If any operation in a transaction fails, the entire transaction is rolled back.

Example:

- Bank transfer: Deduction from Account A succeeds but addition to Account B fails.
- Without Atomicity → ₹1000 is lost.
- With Atomicity → DBMS rolls back deduction, both accounts remain unchanged.

Implementation:

- **Transaction Log** (keeps record of operations).
- **Rollback** (UNDO changes if transaction fails).

(B) Consistency – “Valid State to Valid State”

- Ensures that a transaction takes the database from one **consistent state** to another **consistent state**.
- Database rules, integrity constraints, and triggers must hold true before and after the transaction.

Example:

- A bank rule: Total balance of all accounts = ₹1,00,000.
- After a transfer, the total should remain ₹1,00,000.
- If balance constraints are violated, the transaction is aborted.

Implementation:

- **Integrity Constraints** (Primary Key, Foreign Key, Check).
- **Business Rules** (e.g., salary cannot be negative).

(C) Isolation – “Transactions Do Not Interfere”

- Ensures that **concurrent transactions** are executed as if they were run **serially (one after another)**.
- No transaction can see intermediate results of another transaction.

Problems Without Isolation:

1. **Dirty Read** → Reading uncommitted changes of another transaction.
2. **Unrepeatable Read** → Same query gives different results if another transaction modifies data in between.
3. **Phantom Read** → New rows appear/disappear when re-executing a query.

Example:

- Transaction T1: Transfers money from A → B.
- Transaction T2: Reads Account A's balance while T1 is still in progress.
- Without isolation, T2 may see inconsistent data.

Implementation:

- **Concurrency Control** techniques:
 - Locking (Shared Lock, Exclusive Lock, 2-Phase Locking).
 - Timestamp Ordering.
 - Isolation Levels in SQL (Read Uncommitted, Read Committed, Repeatable Read, Serializable).

(D) Durability – “Once Committed, Forever Saved”

- Ensures that once a transaction is committed, its results are **permanently stored** in the database, even if the system crashes immediately afterward.

Example:

- If you transfer ₹1000 and the system crashes right after showing “Transaction Successful”:
 - Without Durability → Money may disappear.
 - With Durability → DBMS guarantees update is saved to disk.

Implementation:

- **Write-Ahead Logging (WAL):** Log changes before applying them.
- **Checkpoints:** Save database state at intervals.
- **Recovery Manager:** Reapplies changes after crashes.

Why ACID Properties are Important?

- Prevent **data loss** (Atomicity).
- Ensure **data correctness** (Consistency).
- Allow **safe concurrent access** (Isolation).
- Guarantee **permanent updates** (Durability).
- Widely used in **banking, e-commerce, airline reservations, healthcare databases**.

Transaction Management Components

1. Transaction Manager (TM):

- Coordinates transaction execution.
- Ensures ACID properties.

2. Scheduler:

- Orders operations of concurrent transactions.
- Prevents conflicts and ensures serializability.

3. Concurrency Control Manager:

- Handles simultaneous transactions without conflicts.

4. Recovery Manager:

- Ensures durability.
- Uses **logs, checkpoints, rollback, and redo operations.**

Concurrency Control

- Needed when multiple transactions run **simultaneously.**

Problems Without Control:

- **Lost Update Problem**
- **Dirty Read Problem**
- **Unrepeatable Read Problem**
- **Phantom Read Problem**

Techniques:

1. Lock-based Protocols:

- Shared Lock (Read), Exclusive Lock (Write).
- Two-Phase Locking (2PL): Growing phase (acquire locks), shrinking phase (release locks).

2. Timestamp Ordering Protocols:

- Assign timestamps to transactions, execute in order.

3. Optimistic Concurrency Control:

- Assume no conflicts, validate before commit.

Transaction Recovery

• Recovery Techniques:

- **Deferred Update:** Changes applied only after commit.
- **Immediate Update:** Changes applied instantly but logs maintained.

- **Log-based Recovery:**
 - **UNDO** (rollback) → restore before failure.
 - **REDO** (reapply) → ensure committed changes are applied.
- **Checkpointing:**
 - Saves database state at intervals → reduces recovery time.

Advantages of Transaction Management

- Maintains **data consistency**.
- Allows **concurrent access** safely.
- Protects against **system failures**.
- Ensures **integrity of critical applications** (banking, airline booking, e-commerce).

Limitations

- Concurrency control mechanisms may reduce performance.
- Logging and recovery may require high storage and processing.
- Deadlocks can occur (need detection & prevention).

SQL Query Processing

A query in SQL is a request made by the user to retrieve or manipulate data in a relational database.

Query Processing refers to the series of steps performed by the DBMS to transform a high-level SQL query into an efficient execution plan and then run it to produce the required result.

Why Query Processing is Needed?

- SQL queries are written in a high-level declarative language (users describe *what* they want, not *how* to get it).
- DBMS must internally:
 1. Understand the query.
 2. Convert it into low-level operations (scans, joins, sorts).
 3. Optimize execution to minimize cost.

Phases of SQL Query Processing

Query processing happens in **multiple phases**:

1. Parsing and Translation

- SQL query is checked for **syntax and semantics**.
- Tables, attributes, and privileges are verified.
- The query is converted into an **internal representation** (parse tree).

2. Query Rewrite (Logical Optimization)

- The query is rewritten using **algebraic rules** without changing its meaning.
- Example: Push down selections closer to base tables.

SQL: SELECT Name FROM Students WHERE Age > 18;

Relational Algebra: $\pi_{\text{Name}} (\sigma_{\text{Age} > 18} (\text{Students}))$

3. Query Optimization

- DBMS considers **multiple strategies** for executing the query.
- Uses **statistics** (table size, index availability, distribution of values) to estimate cost.
- Chooses the **lowest-cost execution plan**.

4. Execution Plan Generation

- The optimizer produces a **physical execution plan**.
- **Specifies:**
 - Access path (Index Scan, Table Scan).
 - Join method (Nested Loop, Merge Join, Hash Join).
 - Order of operations.

5. Execution

- DBMS executes the plan using **query execution engine**.
- Operators run in an iterator model (pipeline).
- Results are returned to the user.

Importance of Query Processing

- Efficient query processing ensures:
 1. **Performance** → Faster response times.
 2. **Scalability** → Handles large datasets.
 3. **Resource optimization** → Saves CPU, memory, and I/O.
 4. **Reliability** → Ensures correct and consistent results.

Sequence of Execution of an SQL Query:

When you write an SQL query, the typical syntax is:

```
SELECT column_list  
FROM table_name(s)  
WHERE condition  
GROUP BY column(s)  
HAVING condition  
ORDER BY column(s);
```

Actual Execution Order in DBMS

The SQL engine **does not process in SELECT → FROM order**.

Instead, the order is:

1. **FROM** → Identify tables and perform joins.
2. **WHERE** → Filter rows based on conditions.
3. **GROUP BY** → Group remaining rows.
4. **HAVING** → Filter groups based on conditions.

5. **SELECT** → Choose columns/expressions to display.
6. **ORDER BY** → Sort final output.

Step 1: FROM

- DBMS starts with tables in the FROM clause.
- Performs **Cartesian products, joins, or subqueries**.

Step 2: WHERE

- Filters rows before grouping.
- Only rows satisfying the condition remain.

Step 3: GROUP BY

- Groups rows into categories based on column(s).
- Aggregates (COUNT, SUM, AVG, etc.) are computed within each group.

Step 5: SELECT

- Chooses which columns/expressions to display.
- Aggregates are applied here.

Step 6: ORDER BY

- Finally, sorts the result for presentation.

Example:

```
SELECT Dept, COUNT(*) AS Total
FROM Students
WHERE Age > 18
GROUP BY Dept
HAVING COUNT(*) >= 5
ORDER BY Total DESC;
```

Execution Flow:

1. **FROM Students** → Take all students.
2. **WHERE Age > 18** → Keep only students older than 18.
3. **GROUP BY Dept** → Group students by department.
4. **HAVING COUNT(*) >= 5** → Keep only departments with ≥ 5 students.

5. **SELECT Dept, COUNT(*)** → Project columns.

6. **ORDER BY Total DESC** → Sort departments by student count.

SQL Query Optimization

It is the process of choosing the most efficient way to execute a SQL query.

A single SQL query can be executed in many different ways (different join orders, access methods, indexes, and algorithms), but the goal is always the same:

Minimize response time, CPU usage, memory, and disk I/O.

Types of Query Optimization

1. Heuristic (Rule-Based) Optimization

- Uses **predefined transformation rules** to simplify query.
- Example rules:
 - Push σ (**Selection**) down as close to base table as possible.
 - Perform π (**Projection**) early to reduce columns.
 - Replace $A \cap B$ with $A - (A - B)$.
- Fast but may not always give the best plan.

2. Cost-Based Optimization

- Considers **multiple execution strategies**.
- Estimates the **cost** (CPU, I/O, memory) of each plan using **statistics** (table size, number of rows, index availability, distribution of values).
- Selects the **lowest-cost execution plan**.

Cost Components Considered

1. **I/O Cost** → Reading/writing disk pages.
2. **CPU Cost** → Processing tuples (comparisons, hashing, joins).
3. **Communication Cost** → In distributed databases, data transfer across network.
4. **Memory Usage** → Space needed for sorting, hashing.

Query Optimization Techniques

1. Algebraic Optimization

- Use **relational algebra equivalence rules**:

- Selection splitting:

$$\sigma(A \text{ AND } B)(R) \rightarrow \sigma(A)(\sigma(B)(R))$$

Projection pushing:

$$\pi_{\text{name}}(\sigma_{\text{age} > 18}(\text{Students})) \rightarrow \pi_{\text{name}}(\sigma_{\text{age} > 18}(\pi_{\text{name}, \text{age}}(\text{Students})))$$

Join reordering: $(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$

2. Index Optimization

- Use indexes for filtering and joins when available.
- Example:
 - Without index $\rightarrow$ Table scan on Students.
 - With index on Age $\rightarrow$ Faster retrieval.

3. Join Optimization

- Joins are expensive $\rightarrow$ optimizer selects best method:
 - **Nested Loop Join** (good if one relation is small).
 - **Sort-Merge Join** (good for pre-sorted inputs).
 - **Hash Join** (good for large relations, equality joins).
- Optimizer also decides **join order** (which pair to join first).

Query Rewriting

- Subqueries are transformed into joins for efficiency.

Example:

```
SELECT Name
FROM Students
WHERE SID IN (SELECT SID FROM Enrollments WHERE CourseID =
'DBMS101');
```

Optimized:

```
SELECT S.Name
FROM Students S JOIN Enrollments E
ON S.SID = E.SID
WHERE E.CourseID = 'DBMS101';
```

Role of Statistics

The **query optimizer uses metadata** (stored in system catalog):

- Number of rows in table.
- Number of distinct values in a column.
- Distribution histograms of values.
- Indexes available.

Benefits of Query Optimization

1. **Faster query execution** (low response time).
2. **Reduced resource consumption** (CPU, I/O, memory).
3. **Scalability** → supports large datasets.
4. **Better concurrency** → optimized queries reduce lock times.
5. **Cost savings** → important for cloud-based DB systems.

Limitations

- Optimization may not always find the **absolute best** plan (search space too large).
- Depends heavily on **accurate statistics**.
- Sometimes **optimizer's choice is suboptimal** (especially with complex queries or outdated stats).

Concurrency Control

In a multi-user database environment, many transactions run simultaneously. This improves system throughput and resource utilization, but it also creates risks:

- Conflicts in accessing the same data item.
- Possibility of data inconsistency.
- Issues like lost updates, dirty reads, and deadlocks.

Objectives of Concurrency Control

1. **Correctness of Transactions** – Ensure only valid results are produced.
2. **Isolation** – Transactions execute as if they are the only ones in the system.
3. **Serializability** – Guarantee that concurrent transactions are equivalent to some serial schedule.
4. **Consistency** – Final state of the database must satisfy all integrity constraints.
5. **Efficiency** – Maximize system throughput and minimize response time.

Problems in Concurrency

When multiple transactions execute at the same time without control, several problems can occur:

1. Lost Update Problem

- Two transactions update the same data item, and one update is lost.
- Example:
 - T1 reads balance = 5000.
 - T2 also reads balance = 5000.
 - T1 adds 1000 → balance = 6000.
 - T2 subtracts 2000 → balance = 3000 (ignoring T1's update).

2. Temporary (Dirty) Read Problem

- A transaction reads uncommitted changes of another transaction.
- Example:
 - T1 updates salary = 70000 but crashes before commit.
 - T2 reads salary = 70000 → wrong, because T1 never committed.

3. Unrepeatable Read Problem

- A transaction reads the same data twice and gets different results.
- Example:
 - T1 reads salary = 50000.
 - T2 updates salary = 60000.
 - T1 reads salary again → gets 60000.

4. Phantom Problem

- A transaction re-executes a query but sees new rows.
- Example:
 - T1 executes: `SELECT * FROM Employees WHERE dept='HR'`.
 - T2 inserts a new employee in HR dept.
 - T1 executes query again → sees extra rows.

Serializability

- Serial Schedule: Transactions execute one after another, no interleaving.
- Serializable Schedule: A non-serial schedule that produces the same result as a serial schedule.

Types:

1. Conflict Serializability – Based on conflicts between operations.
2. View Serializability – Based on equivalence of read/write operations.

Serializability is the gold standard for concurrency control.

Concurrency Control Techniques

Concurrency control is implemented using several techniques:

1. Lock-Based Protocols

- Lock: A mechanism to restrict access to a data item.
- Types of locks:
 - Shared (S) Lock → Read-only access.
 - Exclusive (X) Lock → Read + Write access.

Two-Phase Locking (2PL) Protocol

- Every transaction has two phases:
 1. Growing Phase – Acquire locks, no release.
 2. Shrinking Phase – Release locks, no acquire.
- Guarantees conflict serializability.
- Variants:
 - Strict 2PL → Locks released only after commit.
 - Rigorous 2PL → All locks released together.

Problems with Locking:

- Deadlock
- Starvation

2. Timestamp-Based Protocols

- Each transaction is assigned a timestamp (TS) when it starts.
- Order of conflicting operations is decided by timestamps.
- Rules:
 - If a transaction tries to read/write an item that violates timestamp ordering → it is rolled back.
- Advantage: No deadlocks.
- Disadvantage: Frequent rollbacks possible.

3. Optimistic Concurrency Control (OCC)

- Assumes conflicts are rare.
- Phases:
 1. Read Phase → Transaction executes without restrictions.
 2. Validation Phase → Before commit, system checks for conflicts.
 3. Write Phase → If validation successful, changes applied.
- Advantage: Good for low-contention environments.
- Disadvantage: Many rollbacks under heavy contention.

4. Mult version Concurrency Control (MVCC)

- Multiple versions of data are kept.
- Readers can access old versions → no waiting for writers.
- Used in PostgreSQL, Oracle, MySQL (InnoDB).

Advantage: High concurrency.

Disadvantage: More storage overhead.

Deadlock in Concurrency Control

- Deadlock: A situation where two or more transactions wait for each other's locks indefinitely.

Example:

- T1 locks A, waits for B.
- T2 locks B, waits for A.

Deadlock Handling

1. Deadlock Prevention
 - Wait-Die scheme (older waits, younger dies).
 - Wound-Wait scheme (older preempts younger).
2. Deadlock Detection
 - Construct a Wait-for Graph.
 - Detect cycles.
3. Deadlock Recovery
 - Abort one transaction to break cycle.

Real-World Examples

- Banking System – Multiple ATMs updating account balances.
- E-commerce Websites – Multiple users purchasing the last product.
- Airline Reservation – Multiple agents booking the last available seat.

Recovery Techniques:

In a database system, failures are unavoidable — they can occur due to hardware crashes, power failures, software bugs, or transaction errors. If a failure occurs while transactions are executing, the database can be left in an inconsistent state.

Recovery Techniques are mechanisms provided by DBMS to restore the database to a consistent state after a failure, while ensuring that the ACID properties of transactions are not violated.

Need for Recovery

1. Atomicity Requirement

- A transaction is either fully executed or not executed at all.
- If failure occurs in the middle, DBMS must rollback changes.

2. Durability Requirement

- Once a transaction commits, its results must be permanent even after a crash.

3. Consistency Requirement

- The database must always satisfy integrity constraints.

Types of Failures

Recovery techniques depend on the type of failure:

1. Transaction Failures

- **Logical Errors** → Transaction cannot complete due to invalid input or programming bug.
- **System Errors** → Transaction aborted due to deadlock or resource shortage.

2. System Crashes

- Main memory content is lost due to power failure or OS crash.
- Disk contents remain intact.

3. Disk Failures

- Media failure such as head crash, bad sectors, or disk corruption.
- Requires restoration from backups.

4. Communication Failures

- Transactions distributed across networks may fail due to message loss.

Database Recovery Techniques

Recovery can be broadly classified into two categories:

1. Backup-Based Recovery

- Restore the database using periodic backups.
- Useful for catastrophic disk failures.

2. Log-Based Recovery

- Use a transaction log to undo/redo operations.
- Suitable for system crashes and transaction failures.

Log-Based Recovery:

A log file is a sequential record of all database operations.

Each log entry contains:

<Transaction-ID, Data-item, Old-value, New-value>

Types of Logging:

1. **Undo Logging** → Used when immediate updates are allowed.
2. **Redo Logging** → Used when updates are deferred until commit.
3. **Undo/Redo Logging** → Combination of both (used in modern DBMS).

1. Deferred Update (No-Undo/Redo)

- Updates are recorded in log but **not applied to database** until transaction commits.
- If system crashes before commit → No action needed.
- If system crashes after commit → Use log to redo.

Advantages:

- Simple and safe (no undo needed).
- Ensures atomicity easily.

Disadvantages:

- Higher response time (changes visible only after commit).

2. Immediate Update (Undo/Redo)

- Updates are applied to the database **as soon as they are made**.
- A log entry is written **before** the update (write-ahead logging).
- If system crashes:
 - For committed transactions → Redo their updates.
 - For uncommitted transactions → Undo their updates.

Advantages:

- Faster response (changes visible immediately).

Disadvantages:

- Complex recovery process (both undo & redo needed).

Checkpointing

- A **checkpoint** is a snapshot of the database and transaction log at a certain point in time.
- During recovery, DBMS does not start from the beginning of the log, but from the **last checkpoint**.

Steps in Checkpointing:

1. Stop accepting new transactions temporarily.
2. Flush all modified buffers to disk.
3. Write a <Checkpoint> record in the log.
4. Resume normal operations.

Advantages:

- Reduces recovery time.
- Prevents scanning of entire log.

Recovery Algorithms:

1. Shadow Paging

- Instead of overwriting data pages, DBMS maintains **two copies**:
 1. **Shadow Page** – The committed, stable copy.
 2. **Current Page** – Modified by ongoing transactions.
- On commit → Current page becomes new shadow.
- On crash → Shadow page used for recovery.

Advantages:

- Simple, no undo required.

Disadvantages:

- High overhead (maintaining multiple copies).
- Not scalable for large databases.

2. ARIES (Algorithms for Recovery and Isolation Exploiting Semantics)

- Widely used in commercial DBMS (IBM DB2, SQL Server, Oracle).
- Based on **Write-Ahead Logging** and **Checkpoints**.
- Phases:
 1. **Analysis Phase** → Identify transactions active during crash.
 2. **Redo Phase** → Redo operations of committed transactions.
 3. **Undo Phase** → Rollback incomplete transactions.

Advantages:

- Efficient and reliable.
- Supports fine-grained recovery.

Recovery in Distributed Databases

- Recovery becomes more complex due to multiple sites.
- **Two-Phase Commit Protocol (2PC):**
 1. **Prepare Phase** → Coordinator asks all sites to prepare to commit.
 2. **Commit Phase** → If all agree, commit is executed; otherwise rollback.
- Ensures atomicity across distributed systems.

Real-World Examples

- **Banking System** – If system crashes during fund transfer, recovery ensures either:
 - Both debit & credit are done, or
 - Neither is done.
- **E-commerce Website** – During order placement, if payment succeeds but order fails, recovery ensures consistency.
- **Airline Reservation** – If system fails after seat allocation but before payment confirmation, recovery releases the seat.